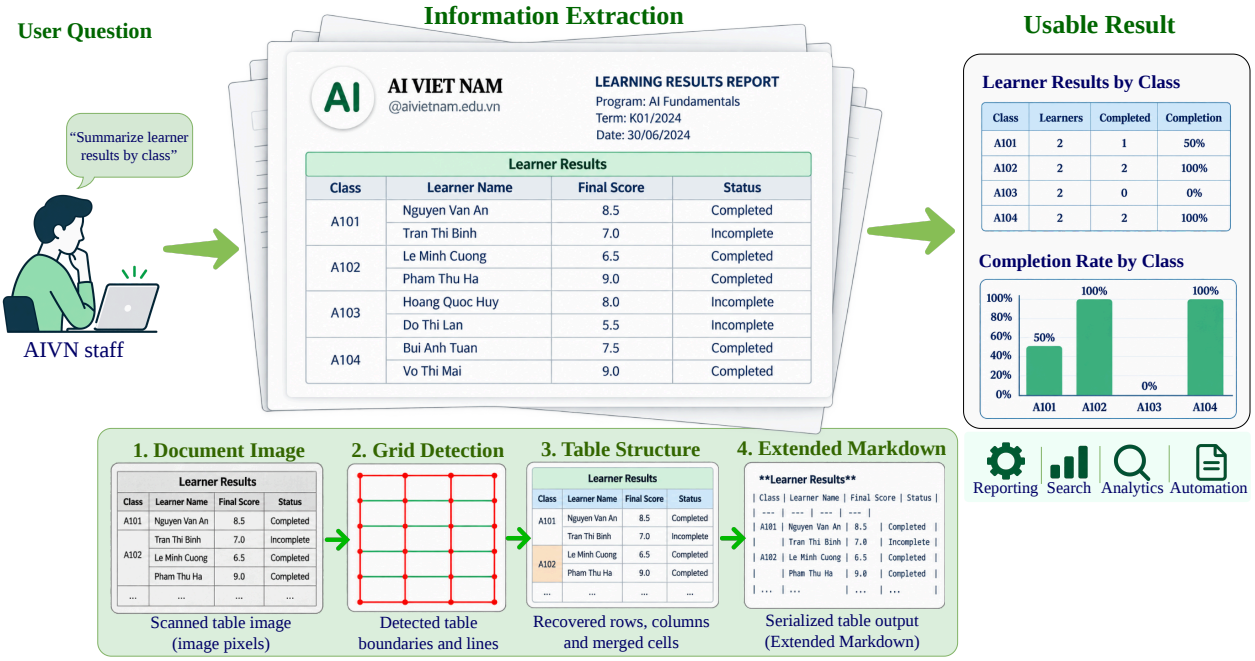
Tutorial: Xây dựng hệ thống trích xuất bảng từ ảnh tài liệu (Olympic AI)

Trần Quang Minh Đinh Quang Vinh

AI VIETNAM xin chân thành cảm ơn trường: Học viện Công nghệ Bưu chính Viễn thông (PTIT) đã cho phép sử dụng đề thi cuộc thi OAI làm dữ liệu cho project này. Thông tin về cuộc thi và bộ dữ liệu có tại Khoa AI - PTIT và VOAI.

I. Giới thiệu

Trong nhiều hoạt động hằng ngày, con người phải đọc bảng trong báo cáo, hóa đơn, hồ sơ, hợp đồng và biểu mẫu hành chính để lấy ra thông tin cần thiết. Khi xử lý thủ công, người đọc phải xác định từng hàng, từng cột, nhận biết các ô được gộp rồi nhập hoặc chuyển thông tin sang hệ thống khác. Công việc này mất nhiều thời gian khi số lượng tài liệu tăng lên và dễ xảy ra sai sót nếu bảng có cấu trúc phức tạp hoặc kéo dài qua nhiều trang. Vì bản quét chỉ tồn tại dưới dạng các điểm ảnh, phần mềm chưa thể trực tiếp tìm kiếm, tổng hợp hoặc phân tích dữ liệu bên trong bảng.



Hình 1: Từ ảnh tài liệu đến dữ liệu bảng có cấu trúc giúp tìm kiếm, phân tích và tự động hóa.

Từ nhu cầu đó hình thành bài toán **Document Image Table Extraction**, tức trích xuất bảng từ ảnh tài liệu. Hệ thống nhận vào một hoặc nhiều ảnh trang và cần khôi phục bảng dưới dạng dữ liệu có cấu trúc. Công nghệ nhận dạng ký tự quang học (**Optical Character Recognition**, viết tắt là **OCR**) giúp đọc từ và con số trong ảnh. Quá trình trích xuất bảng còn phải xác định vị trí bảng, các hàng, các cột, ô gộp và gán nội dung vào đúng ô. Kết quả cần giữ được cả nội dung và cấu trúc của bảng.

Trong bài thi **Olympic AI PTIT 2026**, mỗi tài liệu gồm một hoặc hai ảnh trang A4 và có thể chứa một hoặc nhiều bảng. Hệ thống cần khôi phục nội dung văn bản, số hàng, số cột, các ô gộp, nội dung nhiều dòng, định dạng in đậm và thứ tự của các bảng. Kết quả được lưu dưới dạng **Extended Markdown**, tức Markdown mở rộng, với các quy ước bổ sung để biểu diễn những đặc điểm mà bảng Markdown thông thường chưa thể hiện đầy đủ.

Mục lục

I. Giới thiệu **1**

II. Phân tích đề thi **4**

1. Thể lệ và yêu cầu của cuộc thi 4
2. Dữ liệu 5
3. Hướng triển khai 9
4. Kiến thức cần có 10

III. Xây dựng lời giải **10**

1. Kiến trúc tổng thể 10
2. Tiền xử lý ảnh 11
3. Phát hiện vùng bảng 13
4. Phục hồi lưới cơ sở 14
5. Phục hồi các ô logic 16
6. Nhận dạng nội dung trong ô 19
7. Khôi phục định dạng in đậm 20
8. Xử lý ở cấp tài liệu 22
9. Sinh Extended Markdown 24
10. Kết quả đánh giá 25

Phụ lục **26**

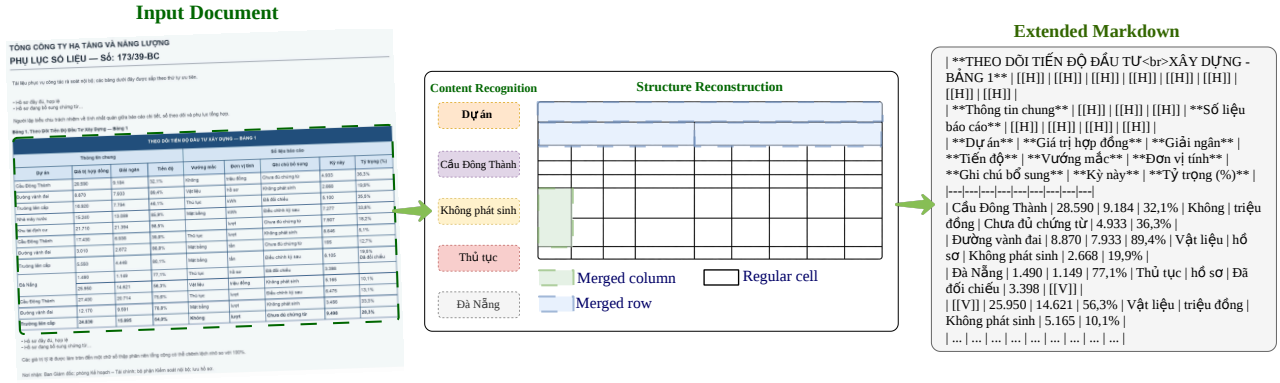
II. Phân tích đề thi

1. Thể lệ và yêu cầu của cuộc thi

Trong bài thi, đơn vị xử lý là một tài liệu. Mỗi tài liệu gồm một hoặc hai ảnh trang A4 và có thể chứa một hoặc nhiều bảng. Hệ thống cần phát hiện đầy đủ các bảng, giữ đúng thứ tự xuất hiện và khôi phục nội dung, số hàng, số cột, các ô gộp, nội dung nhiều dòng và định dạng in đậm.

Các yêu cầu này có thể chia thành hai phần:

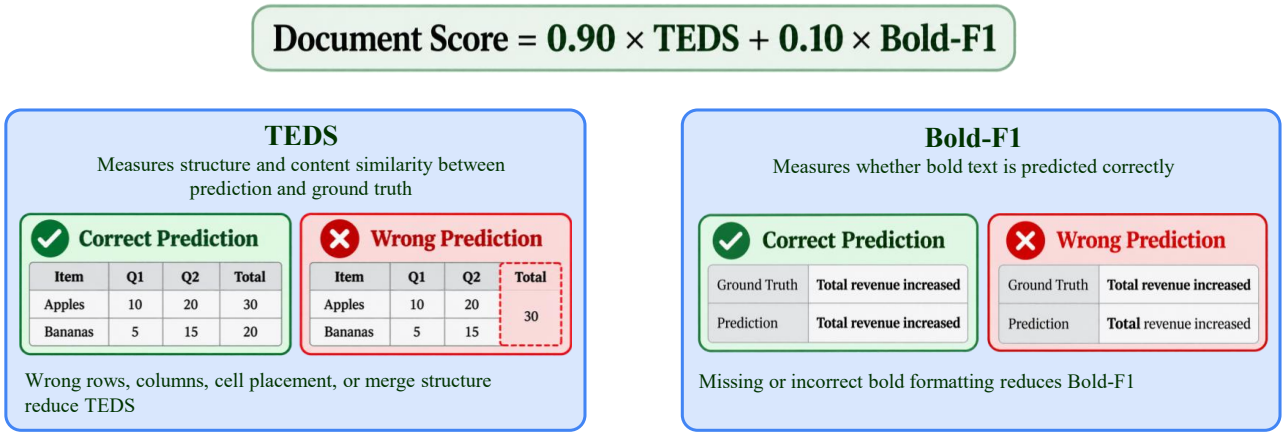
- **Content Recognition**: xác định văn bản xuất hiện trong bảng.
- **Structure Reconstruction**: xác định cách nội dung được tổ chức theo hàng, cột và các ô.



Hình 2: Hai thành phần chính trong quá trình khôi phục bảng từ ảnh tài liệu.

Kết quả được lưu dưới dạng **Extended Markdown**, tức Markdown mở rộng. Bên cạnh cú pháp bảng Markdown thông thường, bài thi sử dụng thêm một số quy ước:

- `[[H]]` đánh dấu phần tiếp tục của một ô được gộp theo chiều ngang.
- `[[V]]` đánh dấu phần tiếp tục của một ô được gộp theo chiều dọc.
- `<br>` biểu diễn xuống dòng bên trong một ô.
- `\|` biểu diễn ký tự `|` xuất hiện trong nội dung.
- `**text**` biểu diễn nội dung in đậm.



Hình 3: Cách TEDS và Bold-F1 đóng góp vào điểm của mỗi tài liệu.

TEDS giữ vai trò chính trong Document Score, nên các lỗi về hàng, cột, vị trí nội dung và quan hệ ô gộp ảnh hưởng đáng kể đến kết quả. Điều này khiến độ chính xác của cấu trúc bảng trở thành một phần quan trọng của lời giải bên cạnh chất lượng nhận dạng văn bản.

Kết quả đầu ra còn phải đáp ứng các điều kiện hợp lệ của bài thi. Một tài liệu sẽ nhận 0 điểm nếu thiếu file kết quả, file rỗng, bảng sai cú pháp hoặc cấu trúc ô gộp không hợp lệ.

2. Dữ liệu

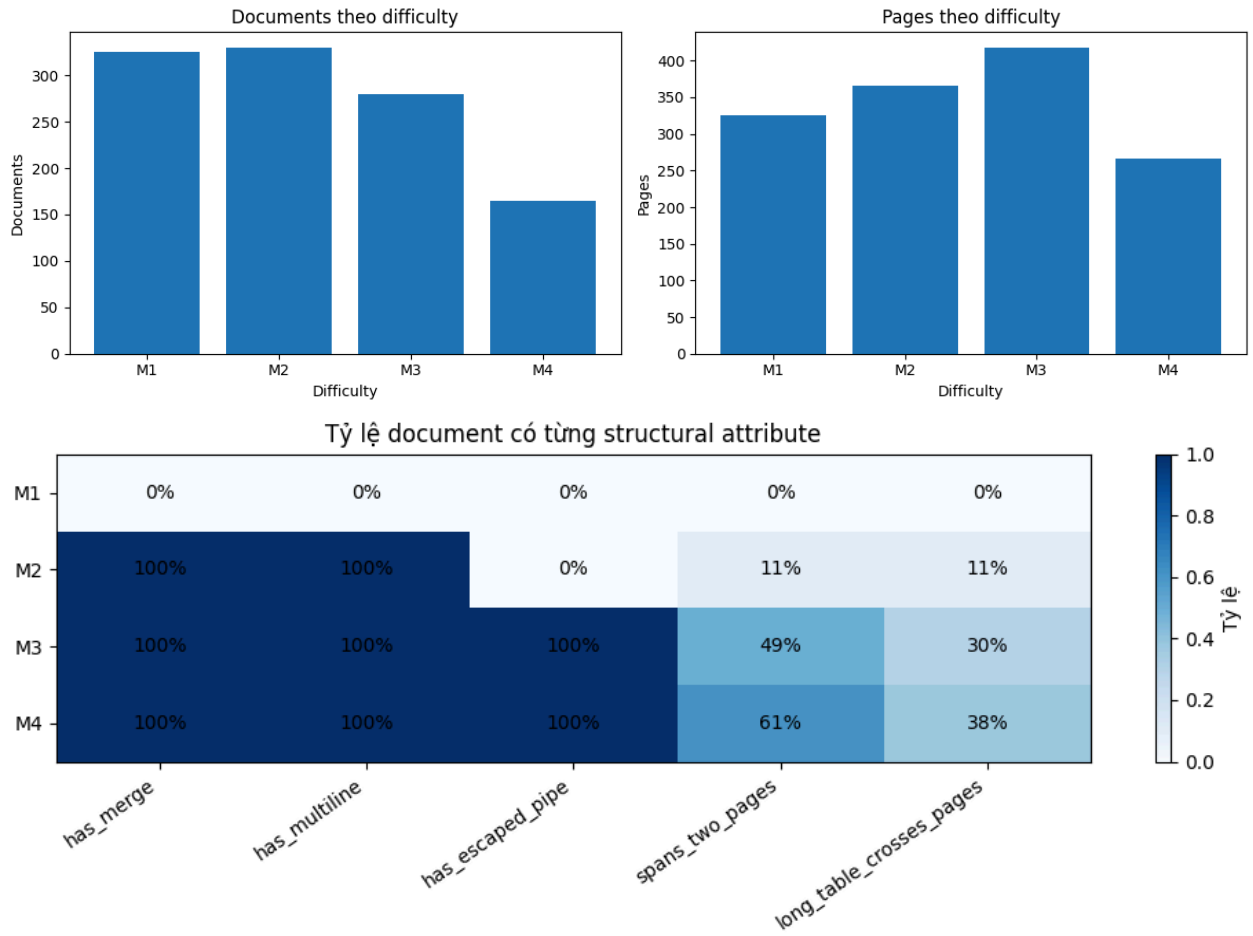
Tập huấn luyện có **1.100 tài liệu**, gồm **1.375 trang** và **1.616 bảng**, được chia thành bốn mức độ khó từ M1 đến M4. Khi độ khó tăng, dữ liệu xuất hiện thêm ô gộp, nội dung nhiều dòng, nhiều bảng trên một trang và bảng kéo dài qua hai trang.

Public test và private test chỉ chứa **M1 và M2**. Vì vậy, hai nhóm này là phạm vi chính khi phân tích dữ liệu và thiết kế baseline.

2.1 Tổng quan dữ liệu và mức độ khó

M1 gồm các bảng có đường kẻ đầy đủ (*full-grid*), mỗi tài liệu chỉ có một bảng trên một trang. Bảng M1 không có ô gộp hoặc nội dung nhiều dòng.

M2 vẫn sử dụng full-grid như M1 nhưng bắt đầu xuất hiện thêm ô gộp, nội dung nhiều dòng, nhiều bảng trên một trang và bảng kéo dài qua hai trang. M3 và M4 mở rộng độ phức tạp hơn nữa, trong đó xuất hiện những bảng không còn hệ thống đường kẻ đầy đủ.



Hình 4: Số tài liệu, trang, bảng và tỷ lệ xuất hiện các đặc điểm cấu trúc theo mức độ khó.

Sự khác biệt về các đặc điểm cấu trúc được phản ánh trong Hình 4. Mọi tài liệu M2 được quan sát đều có ít nhất một ô gộp và một ô chứa nội dung nhiều dòng. Đây là thống kê ở cấp tài liệu, không có nghĩa mọi ô trong bảng đều mang những đặc điểm này.

Độ khó của M2 chủ yếu đến từ quan hệ ô gộp, nội dung nhiều dòng và bố cục nhiều bảng hoặc nhiều trang.

2.2 Bố cục trang và kích thước bảng

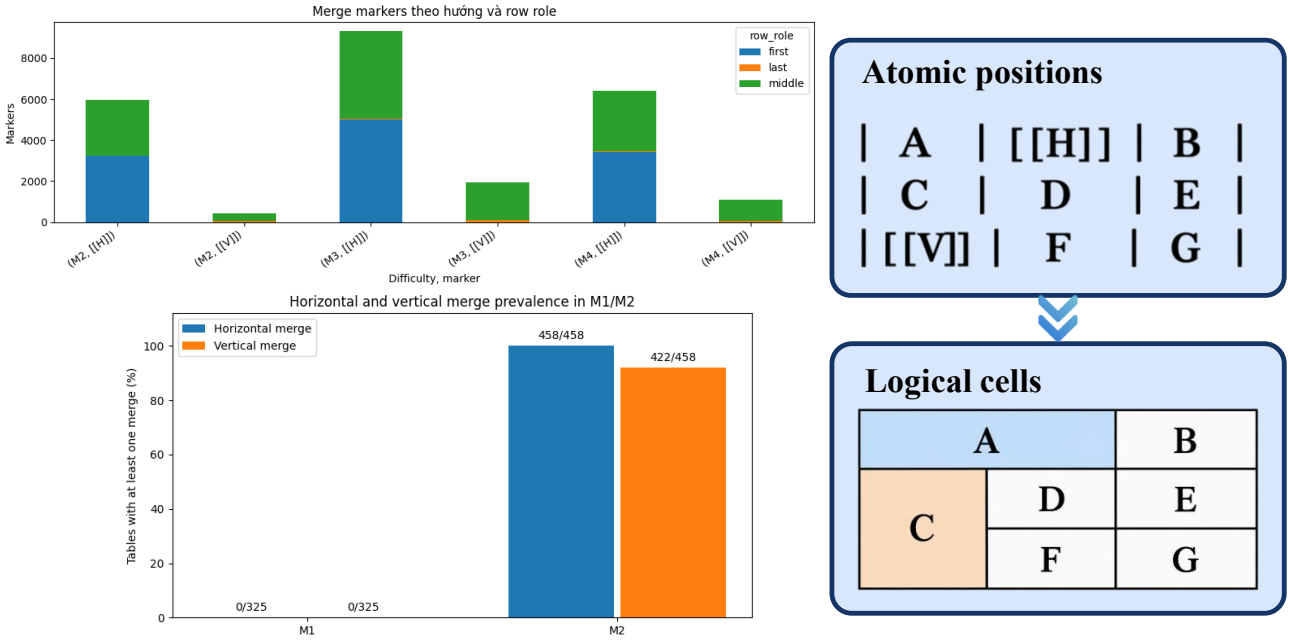
Toàn bộ **325 tài liệu M1** đều có một bảng trên một trang. Trong **330 tài liệu M2**, có **166** tài liệu một bảng trên một trang, **128** tài liệu hai bảng trên cùng một trang và **36** tài liệu có một bảng kéo dài qua hai trang.

Một trang có thể chứa nhiều bảng, nên hệ thống phải phát hiện đầy đủ các vùng bảng và giữ đúng thứ tự của chúng. Với bảng kéo dài qua hai trang, kết quả trên từng trang cần được liên kết lại ở cấp tài liệu.

Số hàng và số cột cũng thay đổi giữa các bảng. M1 có từ **5 đến 7 cột** và **6 đến 9 hàng**. Với M2, số cột nằm trong khoảng **7 đến 9**, còn số hàng dao động từ **11 đến 33**. Vì kích thước thay đổi giữa các bảng, cấu trúc lưới cần được xác định riêng cho từng bảng thay vì dựa trên

một kích thước cố định.

2.3 Ô gộp và đặc điểm nội dung



Hình 5: Tỷ lệ bảng có ít nhất một ô gộp theo chiều ngang hoặc chiều dọc trong M1 và M2.

M1 không có ô gộp. Trong M2, **458/458 bảng** có ít nhất một ô gộp theo chiều ngang và **422/458 bảng** có ít nhất một ô gộp theo chiều dọc. Phân bố này được tóm tắt ở Hình 5.

Các đường hàng và cột trước hết tạo thành một **lưới cơ sở** (*atomic grid*). Một ô thực tế có thể chiếm nhiều vị trí trong lưới này do quan hệ gộp. Vì vậy, việc xác định hàng và cột mới chỉ cho biết cấu trúc hình học cơ sở. Các vị trí liên quan sau đó còn phải được nhóm lại thành **ô logic** (*logical cell*).

Về nội dung, độ dài văn bản trong ô thay đổi không nhiều giữa các mức độ khó. Median text length đều khoảng **5 ký tự**, còn P95 nằm trong khoảng **15 đến 17 ký tự**. Khác biệt đáng chú ý hơn nằm ở nội dung nhiều dòng.

difficulty	cells	bold_rate	multiline_rate	pipe_rate	median_text_length	p95_text_length
M1	12424	26.9%	0.0%	0.0%	5.0	15.0
M2	52573	16.4%	0.9%	0.0%	5.0	16.0
M3	90538	14.1%	1.7%	0.6%	5.0	17.0
M4	72743	11.5%	2.5%	0.4%	5.0	16.0

Example

THPT Phan Bội Châu	12D	38	30	78,9%	23,8%	triệu đồng	3.246 Đã đối chiếu	7.715
THPT Chu Văn An	12E	28	23	82,1%	3,6%	hồ sơ	9.602	372

Hình 6: Tỷ lệ nội dung nhiều dòng và độ dài văn bản trong ô theo từng mức độ khó.

M1 không có ô nhiều dòng, trong khi ở M2 khoảng **0,9% số ô** chứa nội dung nhiều dòng. Chênh lệch này thể hiện rõ trong Hình 6. Tỷ lệ này khá thấp ở cấp ô, nhưng đặc điểm đó xuất hiện trong mọi tài liệu M2 được quan sát.

2.4 Các quy luật được quan sát trong M2

Ngoài những khác biệt về kích thước, ô gộp và nội dung nhiều dòng, M2 còn có một số quy luật ổn định liên quan đến phần đầu bảng, định dạng in đậm và cách bảng tiếp tục qua hai trang.

M2 Header Grammar

Check across M2 page-tables	Result
Row 0 spans **all columns**	494/494
Row 1 splits into two groups at **cols // 2**	494/494
Row 2 contains **individual column** headers	494/494
All three conditions hold together	494/494

Cross-Page Continuation Pattern

Check	Result
Same column count across pages	36/36
First three header rows repeat exactly	36/36
Dropping page 2's **repeated headers** reconstructs the document label	36/36

Bold Grammar

Structural role	Bold cells	Rows entirely bold
Header row 0	100%	494/494
Header row 1	100%	494/494
Header row 2	100%	494/494
Body	0%	0/5,481

Last-row context	Rows entirely bold
One-page terminal	422/422
Page 1	0/36
Page 2	36/36

Hình 7: Các quy luật về cấu trúc phần đầu bảng, định dạng in đậm và phần tiếp tục qua hai trang trong M2.

Trong **494 bảng M2 xuất hiện trên các trang** được kiểm tra, ba hàng đầu đều có cùng cách tổ chức. Hàng đầu tiên trải trên toàn bộ các cột. Hàng thứ hai chia bảng thành hai nhóm, còn hàng thứ ba chứa tiêu đề của từng cột.

Định dạng in đậm cũng gắn với vai trò của hàng. Ba hàng đầu được in đậm, trong khi các hàng dữ liệu thông thường không in đậm. Hàng cuối của bảng chỉ được in đậm khi đó thực sự

là điểm kết thúc của bảng. Vì vậy, hàng cuối trang đầu của bảng hai trang không có cùng định dạng với hàng cuối ở trang thứ hai.

Đối với **36 tài liệu M2 có hai trang**, hai trang có cùng số cột và trang thứ hai lặp lại ba hàng đầu trước khi phần dữ liệu tiếp tục. Khi bỏ phần header lặp lại ở trang thứ hai rồi nối phần còn lại vào trang đầu, nhãn của cả 36 tài liệu đều được khôi phục đúng. Các quy luật này được tổng hợp trong Hình 7.

Các kết quả trên được rút ra từ tập huấn luyện M2, không phải quy tắc chung cho mọi bảng thực tế. Trong phạm vi dữ liệu của bài thi, chúng cung cấp các tín hiệu hữu ích cho việc phục hồi cấu trúc, định dạng và quan hệ giữa hai trang của cùng một bảng.

3. Hướng triển khai

Từ các đặc điểm của M1 và M2, baseline cần xác định cấu trúc bảng, gắn nội dung và định dạng vào từng ô, rồi tạo kết quả theo định dạng mà bài thi yêu cầu. Công việc này gồm bảy phần:

1. **Phát hiện vùng bảng:** Một trang có thể chứa nhiều bảng, nên hệ thống phải xác định đầy đủ các vùng bảng và giữ đúng thứ tự xuất hiện.
2. **Phục hồi lưới cơ sở:** Số hàng và số cột thay đổi giữa các bảng. Ranh giới hàng và cột vì thế cần được xác định từ từng ảnh thay vì dựa trên một kích thước cố định.
3. **Khôi phục các ô logic:** Trong M2, một ô có thể chiếm nhiều vị trí trong lưới cơ sở. Hệ thống cần nhận biết quan hệ ô gộp và nhóm các vị trí tương ứng thành ô logic.
4. **Nhận dạng nội dung trong từng ô:** Sau khi xác định cấu trúc, hệ thống cắt vùng ảnh của từng ô để nhận dạng văn bản bằng OCR và giữ lại nội dung nhiều dòng.
5. **Khôi phục định dạng in đậm:** Định dạng in đậm được chấm điểm riêng. Các quy luật quan sát được trong M1 và M2 cung cấp thêm tín hiệu để xác định những hàng cần giữ định dạng này.
6. **Xử lý ở cấp tài liệu:** Khi một trang chứa nhiều bảng, kết quả phải giữ đúng thứ tự. Với bảng kéo dài qua hai trang, các phần thuộc cùng một bảng cần được nhận biết và nối lại.
7. **Sinh Extended Markdown hợp lệ:** Kết quả cần giữ đúng nội dung, ô gộp, nội dung nhiều dòng và định dạng in đậm, đồng thời tuân theo cú pháp của bài thi.

Baseline sử dụng **OpenCV** để xử lý ảnh và phục hồi cấu trúc bảng, còn **VietOCR** để nhận dạng văn bản. Các quy luật quan sát được trong M1 và M2 hỗ trợ phục hồi ô gộp ở phần đầu bảng, định dạng in đậm và nối các phần của cùng một bảng qua hai trang.

4. Kiến thức cần có

Để theo dõi phần xây dựng lời giải, người đọc nên có nền tảng cơ bản về xử lý ảnh, nhận dạng văn bản và biểu diễn dữ liệu dạng bảng. Các kiến thức này không cần ở mức chuyên sâu, nhưng giúp hiểu vai trò của từng thành phần trong pipeline.

Về lý thuyết, một số nội dung đáng chú ý gồm:

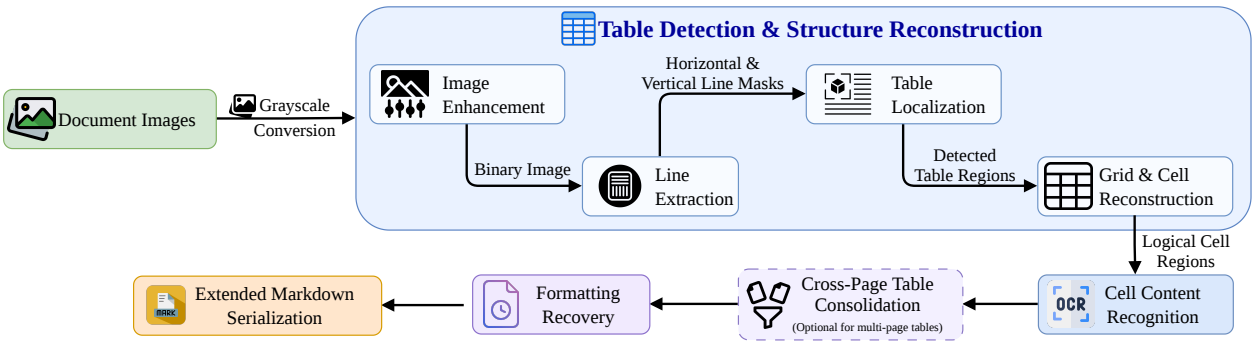
- **Nhận dạng ký tự quang học (Optical Character Recognition, OCR):** hiểu cách văn bản được nhận dạng từ ảnh và cách kết quả nhận dạng được sử dụng trong xử lý tài liệu.
- **Thị giác máy tính cơ bản:** hiểu cách xử lý ảnh và xác định đường kẻ, vùng bảng hoặc vị trí ô trong ảnh tài liệu.
- **Học máy và học sâu cơ bản:** hiểu cách một mô hình đã được huấn luyện được sử dụng để suy luận trên dữ liệu mới.
- **Dữ liệu có cấu trúc:** hiểu cách biểu diễn bảng theo hàng, cột, ô và các quan hệ giữa những ô trong bảng.
- **Đánh giá mô hình:** hiểu cách các chỉ số đánh giá phản ánh chất lượng nhận dạng nội dung, cấu trúc và định dạng của kết quả.

Về thực hành, người đọc nên sử dụng được **Python** và làm quen với các thư viện như **NumPy**, **OpenCV**, **Pillow** và **PyTorch**. **Pandas** và **Matplotlib** cũng hữu ích trong quá trình khám phá dữ liệu và trực quan hóa kết quả.

III. Xây dựng lời giải

1. Kiến trúc tổng thể

Hình 8 trình bày các bước xử lý từ ảnh tài liệu đến Extended Markdown. Ảnh được làm rõ để phát hiện vùng bảng, xác định lưới hàng cột và khôi phục các ô gộp. Sau đó, hệ thống nhận dạng nội dung trong từng ô, bổ sung định dạng in đậm, sắp xếp các bảng theo thứ tự xuất hiện và nối những phần của bảng kéo dài qua hai trang trước khi tạo kết quả.



Hình 8: Kiến trúc tổng thể của hệ thống trích xuất bảng từ ảnh tài liệu.

Cấu trúc bảng được xác định trước khi nhận dạng văn bản. Nhờ vậy, mỗi vùng ảnh đưa vào OCR đã có vị trí tương ứng trong bảng. Nếu ranh giới hàng, cột hoặc quan hệ ô gộp sai, văn bản có thể được đọc đúng nhưng vẫn nằm sai ô trong kết quả.

2. Tiền xử lý ảnh

Ảnh được đọc dưới dạng ảnh xám rồi căn thẳng trước khi dò đường kẻ. Hàm `deskew_gray()` tìm các đường dài trên trang, ước lượng góc nghiêng từ chúng và xoay ảnh về phương ngang, dọc. Nhờ đó, các bước xác định hàng và cột phía sau có đầu vào ổn định hơn.

```python
import re

import cv2
import numpy as np


def deskew_gray(gray: np.ndarray, maximum_degrees: float = 4.0) -> tuple[np.ndarray, float]:
    edges = cv2.Canny(gray, 60, 180)
    height, width = gray.shape
    lines = cv2.HoughLinesP(
        edges,
        1,
        np.pi / 1800,
        threshold=max(70, width // 12),
        minLineLength=max(180, width // 5),
        maxLineGap=max(12, width // 80),
    )

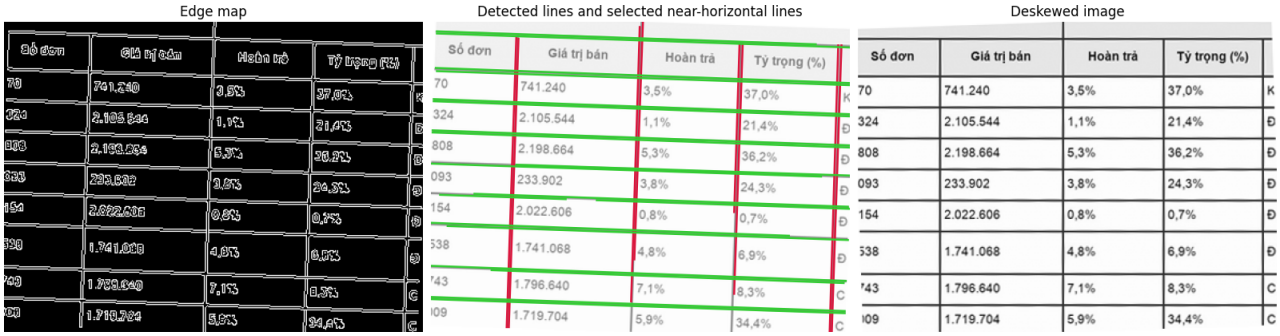
    angles: list[float] = []
    if lines is not None:
        for raw in np.asarray(lines).reshape(-1, 4):
            x0, y0, x1, y1 = map(float, raw)
            angle = float(np.degrees(np.arctan2(y1 - y0, x1 - x0)))
            while angle > 90:
                angle -= 180
            while angle < -90:
                angle += 180
            if abs(angle) <= maximum_degrees:
                angles.append(angle)

    if not angles:
        return gray, 0.0
    angle = float(np.median(angles))
    if abs(angle) < 0.08:
        return gray, 0.0

    matrix = cv2.getRotationMatrix2D((width / 2, height / 2), angle, 1.0)
    rotated = cv2.warpAffine(
        gray,
```

```
        matrix,
        (width, height),
        flags=cv2.INTER_CUBIC,
        borderMode=cv2.BORDER_CONSTANT,
        borderValue=255,
    )
    return rotated, angle


image_path = TRAIN_DIR / "images/train-00460_p01.jpg"
gray = cv2.imread(str(image_path), cv2.IMREAD_GRAYSCALE)
if gray is None:
    raise FileNotFoundError(image_path)
aligned, angle = deskew_gray(gray)
```

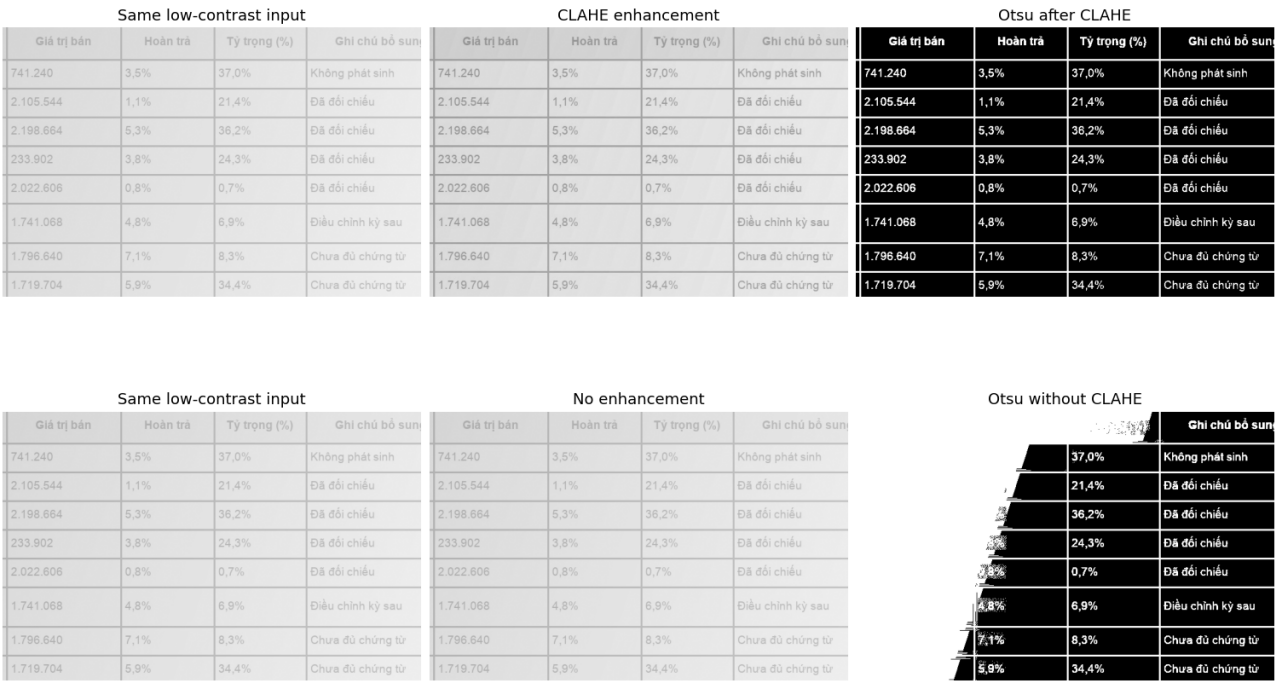


Hình 9: Quá trình phát hiện góc nghiêng và căn thẳng ảnh tài liệu dựa trên các đường kẻ.

Sau khi căn thẳng, CLAHE, một phương pháp tăng tương phản cục bộ, làm rõ chữ và đường kẻ ở từng vùng ảnh. Otsu tiếp tục chọn ngưỡng sáng tối để tạo ảnh nhị phân. Ảnh được đảo màu để nền có giá trị 0, còn chữ và đường kẻ có giá trị 255.

```
contrast = cv2.createCLAHE(
    clipLimit=2.0, tileGridSize=(8, 8)
).apply(aligned)

binary = cv2.threshold(
    contrast, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)[1]
```



Hình 10: CLAHE làm rõ chi tiết trong vùng ảnh giảm tương phản trước khi Otsu tách chữ và đường kẻ khỏi nền.

3. Phát hiện vùng bảng

Từ ảnh nhị phân, phép mở hình thái học giữ lại các đường kẻ theo hai hướng. Một kernel dài theo chiều ngang tạo mask đường ngang, còn kernel cao theo chiều dọc tạo mask đường dọc. Hai mask là đầu vào để xác định vùng bảng và lưới ô.

```python
def line_masks(binary: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    height, width = binary.shape
    horizontal = cv2.morphologyEx(
        binary,
        cv2.MORPH_OPEN,
        cv2.getStructuringElement(
            cv2.MORPH_RECT, (max(45, width // 18), 1)
        ),
    )
    vertical = cv2.morphologyEx(
        binary,
        cv2.MORPH_OPEN,
        cv2.getStructuringElement(
            cv2.MORPH_RECT, (1, max(35, height // 45))
        ),
    )
    return horizontal, vertical


horizontal, vertical = line_masks(binary)
```

Các mask được ghép và làm dày nhẹ để những đoạn kẻ gần nhau tạo thành một vùng liên tục. Contour ngoài của vùng này cho biết khung bao của bảng. Baseline chỉ giữ những vùng đủ rộng, cao và lớn so với trang, rồi sắp xếp chúng theo vị trí xuất hiện.

```
joined = cv2.dilate(
    cv2.bitwise_or(horizontal, vertical),
    cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5)),
    iterations=1,
)
contours, _ = cv2.findContours(
    joined, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

height, width = binary.shape
boxes = []
for contour in contours:
    x, y, w, h = cv2.boundingRect(contour)
    if w >= width * 0.48 and h >= 80 and w * h >= width * height *
        0.012:
         boxes.append((x, y, x + w, y + h))
boxes.sort(key=lambda box: (box[1], box[0]))
```



Hình 11: Từ mask đường kẻ, contour ngoài được chuyển thành khung bao của bảng.

Trong hàm phát hiện hoàn chỉnh, mỗi khung ứng viên còn được kiểm tra số ranh giới hàng, cột và loại bỏ nếu chồng lấn nhiều với một bảng đã giữ.

4. Phục hồi lưới cơ sở

Trong mỗi khung bảng, hệ thống đếm số pixel đường dọc theo từng cột ảnh và số pixel đường ngang theo từng hàng ảnh. Những vị trí có nhiều pixel tạo thành ứng viên ranh giới cột và hàng. Các vị trí gần nhau được gom lại để tránh một đường kẻ dày bị đếm thành nhiều ranh giới.

```
def _runs(mask: np.ndarray, minimum: int = 1, gap: int = 2):
    indices = np.flatnonzero(mask)
    if indices.size == 0:
        return []
    result = []
    start = previous = int(indices[0])
    for raw in indices[1:]:
```

```
        value = int(raw)
        if value - previous > gap:
            if previous - start + 1 >= minimum:
                result.append((start, previous))
            start = value
        previous = value
    if previous - start + 1 >= minimum:
        result.append((start, previous))
    return result


def _line_positions(mask: np.ndarray, axis: int, extent: int):
    projection = np.count_nonzero(mask, axis=axis)
    threshold = max(8, round(extent * 0.42))
    positions = []
    for start, end in _runs(projection >= threshold, gap=3):
        if end - start >= 9:
            positions.extend((start, end))
        else:
            positions.append(round((start + end) / 2))
    return positions


def _dedupe(values: list[int], tolerance: int = 6):
    if not values:
        return []
    groups = [[values[0]]]
    for value in values[1:]:
        if value - groups[-1][-1] <= tolerance:
            groups[-1].append(value)
        else:
            groups.append([value])
    return [round(sum(group) / len(group)) for group in groups]
```

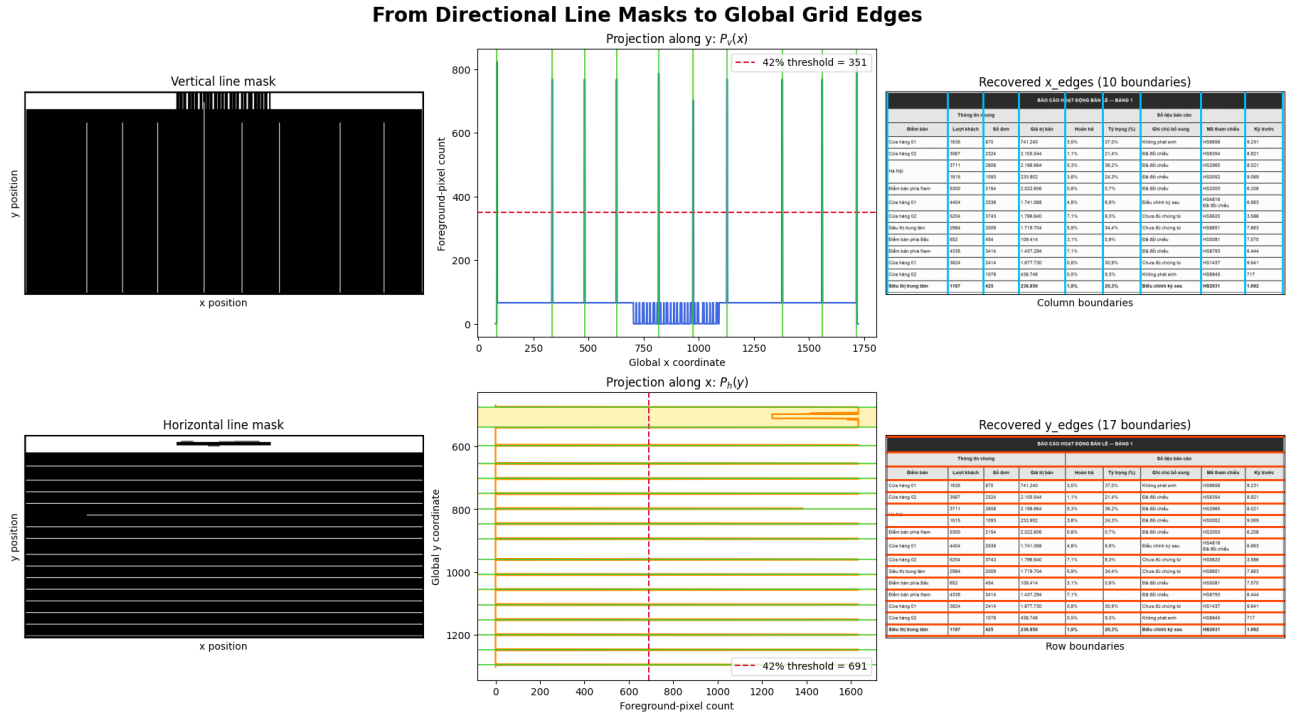
Tọa độ thu được ban đầu nằm trong vùng ảnh đã cắt. Baseline cộng lại độ lệch của khung bảng để đưa chúng về tọa độ trang và bổ sung biên ngoài khi đường kẻ ở mép không được phát hiện đủ rõ.

```
x0, y0, x1, y1 = boxes[0]
pad = 4
roi_x0, roi_y0 = max(0, x0 - pad), max(0, y0 - pad)
roi_x1, roi_y1 = min(width, x1 + pad), min(height, y1 + pad)
local_h = horizontal[roi_y0:roi_y1, roi_x0:roi_x1]
local_v = vertical[roi_y0:roi_y1, roi_x0:roi_x1]

x_edges = _dedupe([
    roi_x0 + value
    for value in _line_positions(local_v, axis=0, extent=roi_y1 -
       roi_y0)
])
y_edges = _dedupe([
```

```
    roi_y0 + value
    for value in _line_positions(local_h, axis=1, extent=roi_x1 -
        roi_x0)
])

if x_edges and abs(x_edges[0] - x0) > 10:
    x_edges.insert(0, x0)
if x_edges and abs(x_edges[-1] - (x1 - 1)) > 10:
    x_edges.append(x1 - 1)
if y_edges and abs(y_edges[0] - y0) > 10:
    y_edges.insert(0, y0)
if y_edges and abs(y_edges[-1] - (y1 - 1)) > 10:
    y_edges.append(y1 - 1)
```



Hình 12: Các line masks được chiếu lên từng trục để phục hồi ranh giới hàng và cột.

Hai ranh giới liên tiếp trên mỗi trục tạo thành một ô nhỏ nhất của lưới cơ sở. Các panel ở cột cuối của Hình 12 đặt những ranh giới vừa tìm được lên ảnh bảng, qua đó thể hiện vị trí các hàng và cột.

Khung bảng cùng các ranh giới này được lưu trong `GridTable`. Bước tiếp theo bổ sung các vùng ô logic vào cấu trúc đó.

5. Phục hồi các ô logic

Lưới cơ sở chia bảng theo hình học, nhưng một ô gộp có thể chiếm nhiều vị trí của lưới. Với M2, baseline nhận diện kiểu bảng từ số cột và chiều cao các hàng đầu, sau đó dùng cấu trúc

ba hàng tiêu đề đã quan sát trong EDA để khôi phục các ô gộp ngang ở phần đầu bảng.

```python
def is_m2_table(table: GridTable) -> bool:
    if table.cols >= 8:
        return True
    if table.cols != 7 or table.rows < 3:
        return False
    heights = np.diff(table.y_edges)
    data_height = float(np.median(heights[2:]))
    return heights[0] / max(1.0, data_height) >= 1.30
```

Với ô gộp dọc, baseline xem đường ngang có thực sự ngăn hai vị trí kề nhau hay không. Tỷ lệ chiều rộng có dấu vết của đường kẻ được đo tại ranh giới giữa hai hàng. Giá trị thấp hơn 0,20 là tín hiệu để ghép hai vị trí đó.

```python
def _segment_coverage(
    mask: np.ndarray, fixed: int, start: int, end: int,
    vertical: bool, radius: int = 2, trim: int = 3,
) -> float:
    start += trim
    end -= trim
    if end <= start:
        return 1.0
    if vertical:
        band = mask[
            start:end,
            max(0, fixed - radius):min(mask.shape[1], fixed + radius +
                1),
        ]
        return float(np.any(band > 0, axis=1).mean())
    band = mask[
        max(0, fixed - radius):min(mask.shape[0], fixed + radius + 1),
        start:end,
    ]
    return float(np.any(band > 0, axis=0).mean())
```

Các quan hệ gộp được gom thành từng nhóm bằng Union-Find. Một nhóm chỉ trở thành ô logic khi các vị trí của nó tạo thành hình chữ nhật đầy đủ; nhóm không hợp lệ được giữ thành các ô riêng để tránh tạo cấu trúc gộp sai.

```python
def infer_regions(
    table: GridTable, horizontal: np.ndarray, vertical: np.ndarray,
    merge_threshold: float = 0.20,
) -> tuple[CellRegion, ...]:
    count = table.rows * table.cols
    parent = list(range(count))

    def find(node: int) -> int:
```

```
        while parent[node] != node:
            parent[node] = parent[parent[node]]
            node = parent[node]
        return node

    def union(left: int, right: int) -> None:
        left, right = find(left), find(right)
        if left != right:
            parent[right] = left

    def node(row: int, col: int) -> int:
        return row * table.cols + col

    if is_m2_table(table):
        for col in range(table.cols - 1):
            union(node(0, col), node(0, col + 1))
        split_col = table.cols // 2
        for col in range(split_col - 1):
            union(node(1, col), node(1, col + 1))
        for col in range(split_col, table.cols - 1):
            union(node(1, col), node(1, col + 1))

    for row in range(table.rows - 1):
        for col in range(table.cols):
            coverage = _segment_coverage(
                horizontal, table.y_edges[row + 1],
                table.x_edges[col], table.x_edges[col + 1],
                vertical=False,
            )
            if coverage < merge_threshold:
                union(node(row, col), node(row + 1, col))

    components: dict[int, list[tuple[int, int]]] = {}
    for row in range(table.rows):
        for col in range(table.cols):
            components.setdefault(find(node(row, col)), []).append((row
                , col))

    regions = []
    for members in components.values():
        rows, cols = zip(*members)
        row0, row1 = min(rows), max(rows) + 1
        col0, col1 = min(cols), max(cols) + 1
        if len(members) == (row1 - row0) * (col1 - col0):
            regions.append(CellRegion(row0, row1, col0, col1))
        else:
            regions.extend(
                CellRegion(row, row + 1, col, col + 1)
                for row, col in members
            )
    return tuple(sorted(regions, key=lambda region: (region.row0,
        region.col0)))
```

Mỗi ô logic có một vị trí đầu tiên để chứa văn bản. Những vị trí còn lại được đánh dấu là phần

tiếp tục của ô gộp ngang hoặc dọc. Cách biểu diễn này giữ nguyên kích thước ma trận ô khi tạo Extended Markdown.

```python
def merge_markers(table: GridTable) -> list[list[str]]:
    cells = [["" for _ in range(table.cols)] for _ in range(table.rows)
        ]
    for region in table_regions(table):
        for row in range(region.row0, region.row1):
            for col in range(region.col0, region.col1):
                if (row, col) != (region.row0, region.col0):
                    cells[row][col] = "[[H]]" if row == region.row0
                        else "[[V]]"
    return cells
```

6. Nhận dạng nội dung trong ô

Mỗi ô logic xác định một vùng ảnh hoàn chỉnh, kể cả khi ô trải qua nhiều hàng hoặc cột của lưới cơ sở. Baseline cắt lùi vào trong hai pixel ở mỗi cạnh để đường viền bảng không đi vào ảnh đưa cho OCR.

```python
table = detect_grid_tables(aligned)[0]
regions = table_regions(table)
cell_crops = []

for region in regions:
    x0 = table.x_edges[region.col0] + 2
    x1 = table.x_edges[region.col1] - 2
    y0 = table.y_edges[region.row0] + 2
    y1 = table.y_edges[region.row1] - 2
    cell_crops.append(aligned[y0:y1, x0:x1])
```

Trong một ô có nhiều dòng, baseline tìm các dải chữ theo chiều dọc và đọc từng dải riêng. Bước đóng ảnh nhỏ theo chiều dọc giúp dấu tiếng Việt nối với thân chữ trước khi xác định các dòng.

```python
def split_text_bands(crop: np.ndarray, min_height: int = 4) -> list[np.
    ndarray]:
    if crop.size == 0:
        return [crop]
    if crop.ndim == 3:
        crop = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
    ink = cv2.threshold(
        crop, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
    )[1]
    border = max(2, round(min(crop.shape) * 0.04))
    ink[:border] = 0
    ink[-border:] = 0
    ink[:, :border] = 0
```

```
    ink[:, -border:] = 0
    ink = cv2.morphologyEx(
        ink, cv2.MORPH_CLOSE,
        cv2.getStructuringElement(cv2.MORPH_RECT, (1, 3)),
    )
    active = np.count_nonzero(ink, axis=1) >= max(
        1, round(crop.shape[1] * 0.01)
    )
    bands = _runs(active, minimum=min_height, gap=3)
    if not bands:
        return [crop]
    result = []
    for start, end in bands:
        y0 = max(0, start - 2)
        y1 = min(crop.shape[0], end + 3)
        result.append(crop[y0:y1, :])
    return result
```

Các dải chữ của mọi ô được gom thành một danh sách để VietOCR nhận dạng theo lô. Kết quả được chia lại cho từng ô, nối bằng dấu xuống dòng trong Extended Markdown và chỉ ghi tại vị trí đầu của ô logic. Các vị trí đánh dấu ô gộp vẫn được giữ nguyên.

```
locations = []
band_counts = []
all_bands = []

for region, crop in zip(regions, cell_crops, strict=True):
    bands = split_text_bands(crop)
    locations.append(region)
    band_counts.append(len(bands))
    all_bands.extend(bands)

texts = recognizer.recognize_images(all_bands)
cells = merge_markers(table)
offset = 0
for region, count in zip(locations, band_counts, strict=True):
    cell_text = texts[offset:offset + count]
    offset += count
    cells[region.row0][region.col0] = "<br>".join(
        text.replace("|", "\\|") for text in cell_text
    )
```

7. Khôi phục định dạng in đậm

Sau OCR, baseline gắn định dạng in đậm theo vai trò của hàng: hàng đầu và hàng cuối của M1, ba hàng đầu của M2. Hàng cuối của M2 cần thêm bằng chứng từ nét chữ, vì một trang có thể kết thúc khi bảng vẫn tiếp tục ở trang sau.

Điểm nét chữ được tính bằng tỷ lệ pixel mực còn lại sau khi ảnh chữ bị co mỏng bằng phép erosion. Điểm này được lưu cho từng ô có nội dung; khi xét hàng cuối M2, baseline lấy trung

vị các điểm hợp lệ của hàng.

```python
M2_LAST_ROW_STROKE_THRESHOLD = 0.08


def bold_stroke_score(crop: np.ndarray) -> float:
    margin = max(3, round(min(crop.shape) * 0.06))
    core = crop[margin:-margin, margin:-margin]
    if core.size == 0:
        return 0.0
    contrast = np.abs(
        core.astype(np.float32) - float(np.median(core))
    ).astype(np.uint8)
    otsu, _ = cv2.threshold(
        contrast, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )
    ink = (contrast >= max(18, otsu)).astype(np.uint8)
    ys, xs = np.nonzero(ink)
    if len(xs) < 8:
        return 0.0
    text_ink = ink[
        int(ys.min()):int(ys.max()) + 1,
        int(xs.min()):int(xs.max()) + 1,
    ]
    area = int(text_ink.sum())
    remaining = cv2.erode(
        text_ink, np.ones((3, 3), dtype=np.uint8), iterations=1
    ).sum()
    return float(remaining) / max(1, area)


def m2_last_row_is_bold(
    table: TableResult, cells: list[list[str]], last_row: int
) -> bool:
    if table.stroke_scores is None:
        return False
    scores = [
        score
        for value, score in zip(
            cells[last_row], table.stroke_scores[last_row], strict=True
        )
        if value not in {"[[H]]", "[[V]]"} and value.strip()
        and score is not None
    ]
    return bool(scores) and float(np.median(scores)) >=
       M2_LAST_ROW_STROKE_THRESHOLD
```

Ký hiệu in đậm chỉ được thêm vào những ô có nội dung. Ô rỗng và các vị trí tiếp tục của ô gộp không bị thay đổi.

```python
BOLD_RE = re.compile(r"^\*\*(.*)\*\*$", re.DOTALL)


def apply_bold_grammar(table: TableResult) -> TableResult:
    cells = [row.copy() for row in table.cells]
    last_row = len(cells) - 1
    is_m2 = is_m2_table(table.grid)
    header_rows = 3 if is_m2 else 1
    bold_last_row = (
        not is_m2 or m2_last_row_is_bold(table, cells, last_row)
    )
    for row_id, row in enumerate(cells):
        if row_id >= header_rows and (
            row_id != last_row or not bold_last_row
        ):
            continue
        for col_id, value in enumerate(row):
            if (
                value in {"[[H]]", "[[V]]"}
                or not value.strip()
                or BOLD_RE.fullmatch(value)
            ):
                continue
            row[col_id] = f"**{value}**"
    return TableResult(table.grid, cells, table.stroke_scores)
```

Điểm nét chữ được tính từ vùng ảnh của từng ô logic rồi lưu cùng ma trận nội dung. Giá trị này được dùng để kiểm tra hàng cuối của bảng M2.

```python
stroke_scores = [[None] * table.cols for _ in range(table.rows)]
for region in regions:
    x0 = table.x_edges[region.col0] + 3
    x1 = table.x_edges[region.col1] - 3
    y0 = table.y_edges[region.row0] + 3
    y1 = table.y_edges[region.row1] - 3
    stroke_scores[region.row0][region.col0] = bold_stroke_score(
        aligned[y0:y1, x0:x1]
    )

table_result = TableResult(table, cells, stroke_scores)
```

8. Xử lý ở cấp tài liệu

Mỗi trang được xử lý thành danh sách các bảng theo thứ tự trên trang. Với tài liệu có hai trang, baseline chỉ thử nối khi mỗi trang có đúng một bảng. Hai bảng phải có cùng số cột và các ranh giới cột tương ứng gần nhau sau khi chuẩn hóa theo chiều rộng bảng.

```python
STITCH_HEADER_ROWS = 3


def _edge_ratios(table: GridTable) -> tuple[float, ...]:
    left, right = table.x_edges[0], table.x_edges[-1]
    width = max(1, right - left)
    return tuple((edge - left) / width for edge in table.x_edges)


def tables_can_stitch(
    first: GridTable, second: GridTable, tolerance: float = 0.025
) -> bool:
    if (
        first.cols != second.cols
        or first.rows < STITCH_HEADER_ROWS
        or second.rows <= STITCH_HEADER_ROWS
    ):
        return False
    return max(
        abs(left - right)
        for left, right in zip(
            _edge_ratios(first), _edge_ratios(second), strict=True
        )
    ) <= tolerance
```

Khi hai phần được nhận là cùng một bảng, ba hàng tiêu đề lặp lại ở trang thứ hai được bỏ trước khi nối. Điểm nét chữ của các hàng còn lại cũng được nối theo để bước khôi phục in đậm dùng được kết quả toàn bảng.

```python
def stitch_two_page_tables(
    first: TableResult, second: TableResult
) -> TableResult | None:
    if not tables_can_stitch(first.grid, second.grid):
        return None
    scores = None
    if (
        first.stroke_scores is not None
        and second.stroke_scores is not None
    ):
        scores = [
            *first.stroke_scores,
            *second.stroke_scores[STITCH_HEADER_ROWS:],
        ]
    return TableResult(
        first.grid,
        [*first.cells, *second.cells[STITCH_HEADER_ROWS:]],
        scores,
    )


results = [table for page in page_results for table in page]
```

```python
if (
    record["page_count"] == 2
    and len(page_results) == 2
    and all(len(page) == 1 for page in page_results)
):
    stitched = stitch_two_page_tables(
        page_results[0][0], page_results[1][0]
    )
    if stitched is not None:
        results = [stitched]
```

9. Sinh Extended Markdown

Sau khi nhận dạng và khôi phục định dạng, ma trận ô đã chứa văn bản, dấu xuống dòng, ký hiệu ô gộp và ký hiệu in đậm. Mỗi hàng được chuyển thành một dòng Markdown, với hàng phân cách được thêm ngay sau hàng đầu tiên.

```python
def _table_markdown(rows: list[list[str]]) -> str:
    lines = ["| " + " | ".join(rows[0]) + " |"]
    lines.append("| " + " | ".join(["---"] * len(rows[0])) + " |")
    lines.extend(
        "| " + " | ".join(row) + " |" for row in rows[1:]
    )
    return "\n".join(lines)
```

Nhiều bảng trong một tài liệu được ngăn bằng một dòng trống. Kết quả được kiểm tra cú pháp trước khi thêm vào danh sách chờ ghi file.

```python
results = [apply_bold_grammar(table) for table in results]
document = (
    "\n\n".join(_table_markdown(table.cells) for table in results)
    if results else EMPTY_TABLE.strip()
)
if not is_valid_markdown(document):
    raise ValueError(f"Kết quả không hợp lệ: {record['id']}")

outputs.append(document.strip() + "\n")
```

Cuối cùng, mỗi kết quả được ghi thành một file Markdown theo ID tài liệu.

```python
write_predictions(PREDICTIONS_DIR, records, outputs)
```

10. Kết quả đánh giá

Sau khi chạy toàn bộ pipeline trên 655 tài liệu M1 và M2 thuộc tập huấn luyện, hệ thống đạt kết quả dưới đây. Các điểm TEDS, Bold-F1 và điểm tổng hợp được trình bày theo thang 100.

Training set

Tập	N	TEDS	Bold-F1	Tổng
M1	325	86,62	100,00	87,96
M2	330	88,46	99,57	89,57
M1/M2	**655**	**87,55**	**99,78**	**88,77**

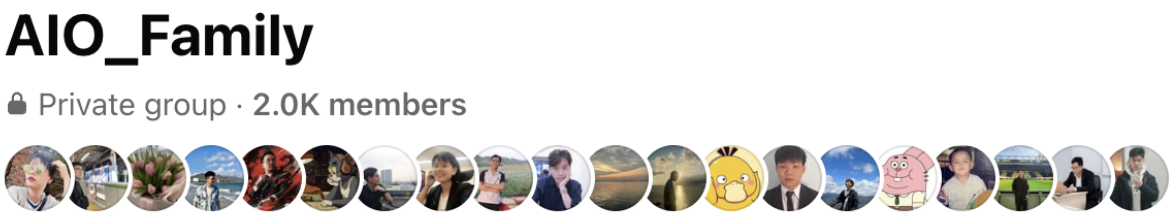
Private test



Bảng 1: Kết quả đánh giá của hệ thống trên tập huấn luyện M1/M2 và private test.

Phụ lục

- **Dữ liệu:** `data.zip`.
- **Mã nguồn:** https://github.com/T-Sunm/olp-ai-ptit-2026-preliminary-round.



Hình 13: Hình ảnh group facebook AIO Q&A.